



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Finding Out What Went Wrong

CMSC 131 Bootcamp Block 4

Prepared by: Rene Andre Bedonia Jocsing

Published: September 02, 2026

Deadline: September 02, 2026

Prerequisites

One archive holds everything this block needs, including a project that already builds. You can start today without Git, without a GitHub account, and without last session's folder.

You need this [archive](#). Unzip it somewhere permanent and work inside the folder it makes.

Session Objectives

- Print the whole register file mid-program with `dump_regs`
- Start your program under `gdb` and stop it where you choose
- Read registers, memory, and upcoming instructions at a breakpoint
- Step one instruction at a time and watch a value change
- Locate the instruction that crashed rather than guessing at it

Scoring

This block is worth 10 points for work completed during its scheduled laboratory session. Your instructor checks your progress before the session ends and prorates the 10 points according to how much of the block you completed. Complete all seven guided blocks, b1 through b7, without an absence and you earn a 30-point completion bonus. Attendance is checked during every bootcamp session, so it doesn't carry a separate score.

Before You Start

You need a working `pc_asm` folder and a `gdb` that runs. Check the second one now, before you need it:

```
1 gdb --version
```

If that prints a version, you're set. If it complains about a shared library it can't identify, your MSYS2 install is half-updated and the debugger is newer than something it depends on. Run `pacman -Syu` in the MSYS2 window until it reports nothing left to do, then try again. The whole block runs on `gdb`, so sort this out first.

The archive holds a working project with today's files in it. If you prefer to keep working in your own folder, copy these across instead.

File	What it's for
<code>broken.asm</code>	A program with one bug in it, which you'll diagnose
<code>broken.input</code>	The divisor 7, for <code>make check</code> once you've fixed it
<code>broken.expected</code>	What the fixed program prints
<code>b3_solution.asm</code>	Block 3's program, finished, if you missed it
<code>print_uint.inc</code>	Needed to build <code>b3_solution.asm</code>

Ninety minutes, roughly fifteen on `dump_regs`, thirty on learning `gdb`, fifteen on reading a crash, and thirty on the exercise.

Part 1: Why This Block Exists

In a language with a runtime, a bug announces itself. Python reports the exception, the line, and the call stack.

Assembly does none of that. Your program prints a wrong number, or prints nothing, or exits with a code and no explanation. There's no line number and no message. The gap between "it's broken" and "here's the instruction that broke it" is yours to close.

Every previous version of this course left students to close it by staring at their source. This block gives you two tools instead.

Part 2: `dump_regs`

Carter's library ships a macro that prints every register at once. It's already available through the `%include` at the top of your file.

```
1 dump_regs 1
```

The number is a label. It lets you tell one dump from another when you have several:

```
1 mov     eax, 100
2 mov     ebx, 7
3 dump_regs 1
4 div     ebx
5 dump_regs 2
```

Running that prints two blocks showing `eax`, `ebx`, `ecx`, `edx`, the index registers, the stack pointers, and the flags, at each point.

The useful way to use `dump_regs` is in pairs around the instruction you doubt, one dump before it and one after it. Then you aren't asking "why is the answer wrong," a question too large to aim at. You're asking "did this one instruction do what I expected," a small one.

Put the dumps around a `div` and you can see immediately whether `edx` was clean going in, which is the single most common cause of a crash in this course.

The library also gives you three relatives, all documented in `asm_io.inc`:

Macro	Shows
<code>dump_regs n</code>	Every general-purpose register and the flags
<code>dump_mem n, addr, k</code>	<code>k</code> paragraphs of memory starting at <code>addr</code>
<code>dump_stack n, a, b</code>	A window of the stack around the current frame

These are debugging scaffolding, so take them out of your submission before you defend, the same way you'd remove stray print statements.

Part 3: gdb

dump_regs requires you to guess in advance where the problem is, rebuild, and look. A debugger lets you stop anywhere and poke around without editing the program at all.

gdb arrived with the toolchain group in Block 1. Confirm it:

```
1  gdb --version
```

Build with the program you want to inspect, then start it under the debugger:

```
1  make PROG=convert
2  gdb convert.exe
```

On Linux, your program has no .exe on the end, so that second line is `gdb ./convert`. The same applies wherever this manual refers to a program to debug. Every gdb command after this point is identical on both platforms.

Two Settings to Apply First

```
1  set disassembly-flavor intel
```

Without this, gdb prints AT&T syntax, where `mov eax, 0x303004` appears as `mov $0x303004,%eax`, with the operands in the opposite order and sigils on everything. You write Intel syntax in NASM, so tell gdb to speak it back to you.

To avoid retyping it every session, put that line in a file called `.gdbinit` in your **home** folder:

```
1  echo "set disassembly-flavor intel" >> ~/.gdbinit
```

gdb refuses to auto-load a `.gdbinit` sitting next to the program it's debugging. That file runs arbitrary commands. gdb has no way to know you wrote it. Put one there and gdb prints a paragraph about auto-load safe-path and then ignores it, which reads like a broken install and isn't one. The copy in your home folder is loaded without complaint.

On Linux, the first gdb of the session may also ask whether to enable debuginfod, which downloads debugging symbols for system libraries. Answer `n`. You're debugging your own code. The question doesn't come back once you've answered it.

Stopping Where You Want

```
1  break asm_main
2  run
```

Notice the mismatch. Your source says `global _asm_main`, with a leading underscore, but the breakpoint above says `asm_main`, without one.

That underscore is a naming convention the 32-bit Windows toolchain applies when C calls into assembly, which is why `driver.c` can declare plain `asm_main` and still find your `_asm_main`. gdb reports the name the way C sees it.

Linux has no such convention and reaches the same place by another route. The four lines you added to `asm_io.inc` in Block 1 strip the underscore before the assembler ever sees it. Either way, the name inside the finished program is `asm_main`, so the breakpoint below is the same one on both platforms.

So: underscore when you declare it in NASM, no underscore when you set a breakpoint. `break _asm_main` answers Function `"_asm_main"` not defined. and is the first thing that confuses people here.

Looking Around

Once stopped, these four commands cover almost everything you need.

```
1  info registers          show every register
2  info registers eax edx  show just the ones you name
3  x/5i $eip              show the next 5 instructions
4  x/4dw &my_label        show 4 decimal words at a label
```

A session looks like this:

```
1  Thread 1 hit Breakpoint 1, 0x00301504 in asm_main ()
2  (gdb) x/4i $eip
3  => 0x301504 <asm_main+4>:  pusha
4      0x301505 <asm_main+5>:  mov     eax,0x303004
5      0x30150a <asm_main+10>: call    0x301564 <print_string>
6      0x30150f <asm_main+15>: call    0x3015a1 <print_nl>
7  (gdb) info registers edx
8  edx                0x770fd9bc        1997529020
```

Look hard at that last line. `edx` already holds 1,997,529,020 before the program has run a single one of your instructions. Nobody put it there on purpose. It's whatever Windows happened to leave behind on the way in.

This is the concrete reason Block 3 insisted on `mov edx, 0` before every `div`. If you skip it, that's your dividend.

Moving Forward

Command	Effect
<code>si</code>	Step one instruction, following calls into them
<code>ni</code>	Step one instruction, stepping over a whole call
<code>continue</code>	Run until the next breakpoint or the end
<code>finish</code>	Run until the current routine returns

Use `ni` over a `call print_string` unless you want to walk through Carter's library. Use `si` on your own code.

Typing at a Program Under `gdb`

Your program still reads the keyboard when it runs under `gdb`. When it stops at a prompt, type the number and press Enter as you normally would. The debugger is watching the program.

`make check` works the other way round, feeding input from a file. Under `gdb` there's no file involved unless you ask for one.

The Whole Command Set, in One Place

Everything this block uses.

Command	What it does
<code>break asm_main</code>	Stop when execution reaches your entry point
<code>run</code>	Start the program
<code>continue</code>	Carry on to the next breakpoint or the end
<code>si</code>	Step one instruction, following a call into it
<code>ni</code>	Step one instruction, stepping over a whole call
<code>finish</code>	Run until the current routine returns
<code>info registers</code>	Show every register
<code>info registers eax edx</code>	Show only the ones you name
<code>x/5i \$eip</code>	Show the next five instructions
<code>x/4dw &label</code>	Show four decimal words stored at a label
<code>set disassembly-flavor intel</code>	Print instructions the way you write them
<code>quit</code>	Leave

Leaving

```
1 quit
```

Part 4: Reading a Crash

When a program dies, run it under gdb and let it die there. gdb stops at the fault.

```
1 gdb convert.exe
```

```
1 (gdb) run
2 Program received signal SIGFPE, Arithmetic exception.
3 0x00401520 in asm_main ()
4 (gdb) x/1i $eip
5 => 0x401520 <asm_main+32>:  div    ebx
6 (gdb) info registers eax edx ebx
```

Three commands and the question is answered. SIGFPE is an arithmetic fault, the faulting instruction is a `div`, and the register dump will show you either a zero divisor or a dirty `edx`.

Two signals account for nearly everything you'll hit in this course:

SIGFPE is a divide error. Either the divisor is zero, or the quotient is too large to fit in `eax`, usually because `edx` is dirty.

SIGSEGV is a bad memory access. Usually a label used as a value where an address was meant, or the other way round.

Part 5: Exercise

You're given a program that crashes. Find out why without editing it first.

```
1 %include "asm_io.inc"
2
```

```

3  segment .data
4  prompt db "Enter a divisor: ", 0
5  result db "1000 divided by your number is: ", 0
6
7  segment .bss
8
9  segment .text
10     global _asm_main
11 _asm_main:
12     enter    0,0
13     pusha
14
15     mov     eax, prompt
16     call    print_string
17     call    read_int
18     mov     ebx, eax
19
20     mov     eax, 1000
21     div     ebx
22
23     mov     ecx, eax
24     mov     eax, result
25     call    print_string
26     mov     eax, ecx
27     call    print_int
28     call    print_nl
29
30     popa
31     mov     eax, 0
32     leave
33     ret

```

It came with the archive as `broken.asm`, so you don't have to type it. Build it with `make PROG=broken`.

1. Run it and enter 7. Record what happens.

What happens is nothing. No prompt, no error, no output at all, and you get your shell back. Sit with that for a second. The program clearly did print a prompt before it reached the division. The output went into a buffer that the C library was holding onto, waiting for a reason to write it out. The crash killed the process before that ever happened. A program that dies can lose output it had already produced. That's why you don't debug by adding print statements alone.

2. Run it again under `gdb`, break at `asm_main`, and step until you reach the `div`.
3. Report the value of `edx` immediately before the `div` executes.
4. Explain in one sentence why entering 7 can fail even though $1000 \div 7$ is perfectly reasonable.
5. Fix it with a single added instruction and confirm with `make PROG=broken check`, which prints OK: `broken matches broken.expected`.
6. Then run it once more and enter 0. Note which signal you get and how it differs from the first fault.

Both failures are a SIGFPE. Only a look tells them apart.

Underneath the signal the two aren't identical. Windows is more specific than `gdb`. Entering 0 asks for a division by zero. The process exits with the status `0xC0000094`. Entering 7 with a dirty `edx` asks for a quotient far too large for `eax` to hold. The status is `0xC0000095`, which Windows calls an integer overflow rather than a divide by zero. Same instruction, same signal, different reason. Only one of them is the bug you were told to look for.

Testing Checklist

Core Functionality

- `dump_regs` prints a labelled register block where you placed it

- `gdb` stops at `break asm_main` and `run`
- `set disassembly-flavor intel` makes the output match the syntax you write
- `x/5i $eip` shows upcoming instructions
- `si` advances exactly one instruction and the register view changes accordingly
- You can identify the faulting instruction in `broken.asm` without editing it
- The one-instruction fix makes `check` pass
- Entering `0` still faults after the fix, and you can say why

Common Pitfalls

- `break _asm_main` with the underscore, which `gdb` doesn't recognise
- Forgetting `set disassembly-flavor intel` and trying to read AT&T operand order
- Using `si` into `print_string` and getting lost in library code, where `ni` was wanted
- Reading registers after the program exited, when `gdb` reports `No registers`.
- Shipping `dump_regs` calls in work you submit

Architecture Review

What We Built

- A habit of bracketing a doubtful instruction with register dumps
- A working `gdb` setup that speaks the syntax you write
- A repeatable procedure for turning a crash into a named instruction
- Direct evidence that registers hold garbage before you initialise them

Key Takeaways

1. **Assembly reports nothing on its own.** Every diagnostic here is one you chose to collect.
2. `dump_regs` **in pairs turns a big question into a small one.**
3. **Declare `_asm_main`, break on `asm_main`.** The underscore is a calling convention.
4. `edx` **holds junk at entry.** You've now seen the number.
5. **One signal, several causes.** `SIGFPE` from a dirty `edx` looks identical to `SIGFPE` from dividing by zero, so inspect rather than assume.

Next Session Preview

- The flags register, and what `cmp` changes
- Conditional jumps, and choosing between them
- Turning `if`, `while`, and `for` into labels and branches
- Why a loop written backwards is often the cleaner one